

IAS Architecture Design

Course Name: EG 212, Computer Architecture - Processor Design

Introduction:

The IAS machine was the first electronic computer built at the Institute for Advanced Study (IAS) in Princeton, New Jersey. It is sometimes called the von Neumann machine, since the paper describing its design was edited by John von Neumann, a mathematics professor at both Princeton University and IAS. The computer was built under his direction, starting in 1946 and finished in 1951. The general organization is called von Neumann architecture, even though it was both conceived and implemented by others. The computer is in the collection of the Smithsonian National Museum of American History but is not currently on display. [1]

The IAS followed a stored-program architecture. It used about a 1000 memory locations all comprising of vacuum tubes. It had a central control unit with an arithmetic logic unit (ALU). Its memory system allowed for random access, as well as dynamically changing parts or the entire word itself.

Assembler:

The Assembler, a python program that converts our assembly file to an object file follows a very basic approach. The few extra commands we added are `jump++` which jumps if the AC is greater than 0, `HALT` to halt the program and `NOP` to signify no code or instruction in that particular memory location or part of that memory location. The assembler acts as a text replacer, and it also ignores all comments. (lines starting with `//`).

Processor:

The Processor, being a little more complex than the assembler, follows the flow chart depicted below. It has a variety of classes owing to the different objects and their requirements. Obviously this could have been more modularized but just for the sake of a project on the IAS ISA it seemed redundant. We have implemented the Main Memory, the IBR, the MAR and the CPU itself as classes with their respective methods. One instruction was giving us a hard time was the jump right instruction. Since the IAS machine only goes to the next instruction when IBR is empty, and IBR is only empty when both instructions are executed, hence we had to set a flag variable that would let the processor know that it only has to execute the right instruction and not both the left and the right instruction at the same time.

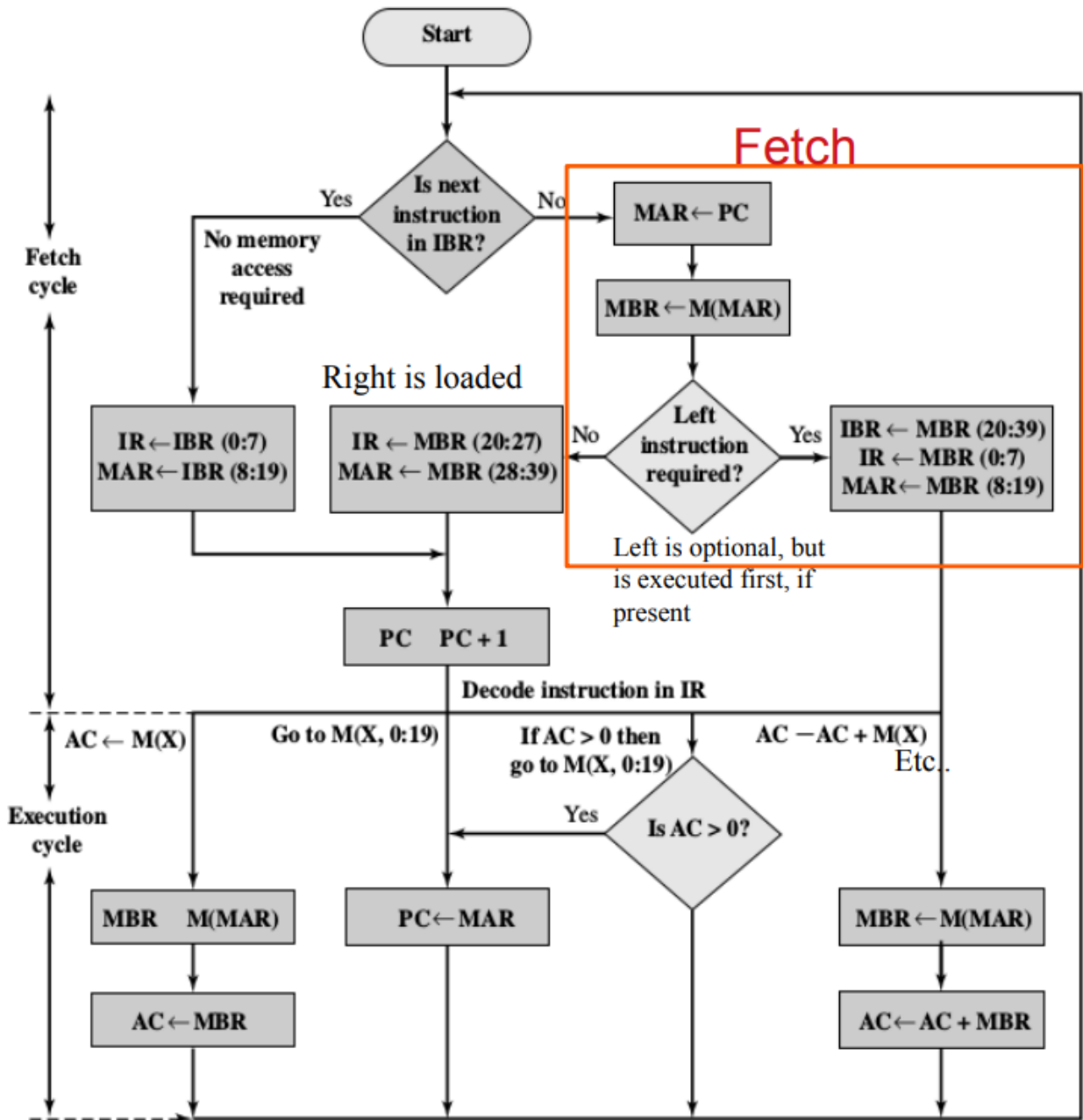


Figure 1: IAS Flow Chart

C++ Equivalent of Assembly Program

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = 5;
    int sum = 0;

    for(int i = 0; i < n; i++) {
        sum += arr[i];
    }

    cout << "Sum = " << sum << endl;
    return 0;
}
```

Assembler Source Code

```
opcode = {
    "NOP": "00000000",
    "LOAD": "00000001",
    "ADD": "00000101",
    "STOR": "00100001",
    "HALT": "11111111"
}

def to_bin(n, bits):
    return format(int(n), f"0{bits}b")

def mem(x):
    return x.replace("M(", "").replace(")", "")

memory = {}

with open("assembly.txt") as f:
    for line in f:
        line = line.strip()
        if not line:
            continue

        if ":" in line and "|" not in line:
            a, v = line.split(":")
            memory[int(a)] = to_bin(v, 40)
            continue
```

```
addr_part, inst_part = line.split(":")
addr = int(addr_part)

left, right = inst_part.split("|")
left = left.strip().split()
right = right.strip().split()

def encode(inst):
    if inst[0] in ("HALT", "NOP"):
        return opcode[inst[0]] + "0"*12
    return opcode[inst[0]] + to_bin(mem(inst[1]), 12)

memory[addr] = encode(left) + encode(right)
```

```
max_addr = max(memory)

with open("binary.txt", "w") as f:
    for i in range(max_addr + 1):
        f.write(memory.get(i, "0"*40) + "\n")

print("Assembler done → binary.txt generated")
```

Processor Source Code

```
PC = 0
AC = 0
MAR = 0
MBR = "0" * 40
IR = "0" * 8
IBR = "0" * 20

def b2i(b):
    return int(b, 2)

def i2b(i):
    return format(i & ((1<<40)-1), "040b")

OP = {
    "LOAD": "00000001",
    "ADD": "00000101",
    "STOR": "00100001",
    "JUMP": "00001101",
    "JUMP+": "00001111",
    "HALT": "11111111"
}

def disk_read(addr):
    try:
        with open("binary.txt", "r") as f:
            lines = f.readlines()
            if addr < len(lines):
                return lines[addr].strip()
            return "0" * 40
    except FileNotFoundError:
        return "0" * 40

def disk_write(addr, val):
    lines = []
    try:
        with open("binary.txt", "r") as f:
            lines = [line.strip() for line in f]
    except FileNotFoundError:
        lines = []

    while len(lines) <= addr:
        lines.append("0" * 40)
    lines[addr] = val

    with open("binary.txt", "w") as f:
        for line in lines:
            f.write(line + "\n")

def get_file_len():
    try:
```

```

        with open("binary.txt", "r") as f:
            return sum(1 for _ in f)
    except FileNotFoundError:
        return 0

def show():
    print("-" * 50)
    print(f"IR -> {IR}")
    print(f"MAR -> {MAR}")
    print(f"MBR -> {MBR}")
    print(f"IBR -> {IBR}")
    print(f"AC -> {AC}")
    print(f"PC -> {PC}")
    print("-" * 50)

print("IAS Processor started")

while True:
    if PC >= get_file_len(): break

    MAR = PC
    MBR = disk_read(MAR)

    LEFT = MBR[:20]
    RIGHT = MBR[20:]

    IR = LEFT[:8]
    MAR = b2i(LEFT[8:])
    IBR = RIGHT
    show()

    if IR == OP["LOAD"]: AC = b2i(disk_read(MAR))
    elif IR == OP["ADD"]: AC += b2i(disk_read(MAR))
    elif IR == OP["STOR"]: disk_write(MAR, i2b(AC))
    elif IR == OP["JUMP"]:
        PC = MAR
        continue
    elif IR == OP["JUMP+"]:
        if AC > 0:
            PC = MAR
            continue
    elif IR == OP["HALT"]:
        break

    IR = IBR[:8]
    MAR = b2i(IBR[8:])
    IBR = "0" * 20
    show()

    if IR == OP["LOAD"]: AC = b2i(disk_read(MAR))
    elif IR == OP["ADD"]: AC += b2i(disk_read(MAR))
    elif IR == OP["STOR"]: disk_write(MAR, i2b(AC))
    elif IR == OP["JUMP"]:
        PC = MAR
        continue
    elif IR == OP["JUMP+"]:
        if AC > 0:
            PC = MAR
            continue
    elif IR == OP["HALT"]:
        break

    PC += 1

print("Sum stored at M(25) =", b2i(disk_read(25)))

```

Binary Output

[illegible][illegible][illegible]

Execution Results

[illegible]

HALT encountered

```
FINAL RESULT:
Sum stored at M(25) = 150
```